

Аннотированный список статей и ресурсов по языку программирования Julia

1. Официальная документация Julia

Название: The Julia Language Documentation

Автор: The Julia Language Community

Ссылка: The, Julia Language Community Julia Documentation / Julia Language Community The. — Текст : электронный // The Julia Language Documentation : [сайт]. — URL: <https://docs.julialang.org/en/v1/> (дата обращения: 18.09.2026).

Аннотация: Официальное руководство по языку Julia, охватывающее все аспекты: от установки и базового синтаксиса до параллельных вычислений и метапрограммирования. Документация структурирована по разделам Manual, Standard Library и Developer Documentation, что позволяет использовать её как справочник и как учебное пособие.

2. Численные решения задачи Таммеса

Название: Numerical Solutions of the Tammes Problem

Автор: L. Hars

Ссылка: Hars, L. Numerical Solutions of the Tammes Problem for up to 60 Points / L. Hars. — Текст : электронный // Numerical Solutions of the Tammes Problem : [сайт]. — URL: https://www.hars.us/Papers/Numerical_Tammes.pdf (дата обращения: 18.09.2026).

Аннотация: Работа посвящена численному решению задачи Таммеса - оптимизационной задачи о размещении N точек на сфере с максимальным минимальным расстоянием между ними. Автор демонстрирует реализацию оптимизационного цикла на Julia с использованием адаптивного управления количеством вычислений целевой функции. Приведённый код включает механизмы прерывания длительных вычислений, обновления оценки вычислительной сложности и верификации результатов. Работа полезна как пример организации вычислительного эксперимента с длительными итерационными процессами.

3. Численные методы и примеры программирования

Название: Numerical Methods: Implementations and Examples

Автор: Коллектив авторов (Springer)

Ссылка: Numerical Methods: Implementations and Examples [Электронный ресурс] // Springer. — URL: <https://link.springer.com/content/pdf/bbm:978-981-97-4631-6/1> (дата обращения: 18.09.2026).

Аннотация: Приложение к учебнику по численным методам, содержащее реализации классических алгоритмов на Julia. Рассматриваются рекурсивное вычисление чисел Фибоначчи, квадратура Симпсона для численного интегрирования, интерполяция Лагранжа и метод Монте-Карло для вычисления числа π . Особую ценность представляет демонстрация работы с массивами неизвестного размера и использования фильтрации для обработки данных.

4. Глоссарий экосистемы пакетов Julia

Название: Glossary — Pkg.jl

Автор: Julia Packaging Community

Ссылка: Glossary [Электронный ресурс] // Pkg.jl Documentation. — URL: <https://pkgdocs.julialang.org/v1/glossary/> (дата обращения: 18.09.2026).

Аннотация: Справочный ресурс, разъясняющий ключевые концепции управления пакетами в Julia: Project, Manifest, Package, Application, Environment, Registry. Описывается различие между пакетами и модулями, структура файлов Project.toml и Manifest.toml, а также механизмы воспроизводимости вычислительных сред. Необходим для понимания организации кода в крупных исследовательских проектах.

Примеры решения задач на Julia

1. Рекурсивное вычисление чисел Фибоначчи

Простейший пример рекурсивной функции на Julia, демонстрирующий синтаксис условных выражений и возврата значений:

```
function fibonacci(n)
    if n == 0 || n == 1
        return 1
    else
        return fibonacci(n - 1) + fibonacci(n - 2)
    end
end
```

Пояснение: Функция принимает целое число `n` и возвращает соответствующее число Фибоначчи. Оператор `||` реализует логическое «или», объединяя проверки граничных условий. Данная реализация наглядна, но вычислительно неэффективна для больших `n` из-за экспоненциальной сложности.

2. Метод Монте-Карло для вычисления числа π

Компактная реализация с использованием генераторов списков и фильтрации:

```
using LinearAlgebra

function monte_carlo_pi(n)
    points1 = [(rand(), rand()) for _ in 1:n]
    points2 = filter(x -> norm(x) <= 1.0, points1)
    return 4 * (length(points2) / n)
end
```

Пояснение: Строка 4 генерирует `n` случайных точек в единичном квадрате. Функция `filter` с анонимным предикатом `x -> norm(x) <= 1.0` отбирает точки, попавшие внутрь единичной окружности. Отношение числа таких точек к общему числу, умноженное на 4, даёт приближение числа π . Использование `norm` из пакета `LinearAlgebra` обеспечивает корректную работу с кортежами координат.

3. Оптимизационный цикл с адаптивным управлением

Фрагмент кода из работы по численному решению задачи Таммеса, демонстрирующий организацию длительных итерационных вычислений:

```
for iter = 1:MAXITERS
    isready(chn1) && break # прерывание по нажатию клавиши
    xinit = init(N, iter, seed) # генерация допустимой начальной популяции
    p = sqrt((1 + xinit[1]) / (1 - xinit[1]))
    opt.lower_bounds = [-1.0; fill(-p, 2N - 4)]
    opt.upper_bounds = [+1.0; fill(+p, 2N - 4)]
    opt.mxeval = ceil(Int, 20 * mxeval) # адаптивный лимит вычислений
    minf, minx, ret = optimize(opt, xinit)
    OK = verify(minx)
    if OK && ret != :FORCED_STOP && ret != :FAILURE
        mxeval = 0.97 * mxeval + 0.03 * opt.numevals # обновление оценки
    end
    # ... вывод и сохранение результатов
end
```

Пояснение: Цикл реализует многократный запуск оптимизации с различными начальными приближениями. Ключевые элементы: (1) проверка `isready` обеспечивает возможность корректного прерывания; (2) адаптивный лимит `mxeval` предотвращает бесполезные вычисления при плохой сходимости; (3) экспоненциальное сглаживание `mxeval` обновляет ожидаемое число вычислений функции на основе успешных запусков. Такой подход типичен для исследовательских вычислительных экспериментов.

4. Решение задачи о рюкзаке через MathOptInterface

Пример формулировки и решения задачи бинарной оптимизации с использованием низкоуровневого интерфейса MathOptInterface:

```
import MathOptInterface as MOI
using GLPK

# Исходные данные
c = [1.0, 2.0, 3.0] # стоимости предметов
w = [0.3, 0.5, 1.0] # веса предметов
C = 3.2             # вместимость рюкзака

optimizer = GLPK.Optimizer()
x = MOI.add_variables(optimizer, length(c))

# Целевая функция: максимизация суммарной стоимости
MOI.set(optimizer,
    MOI.ObjectiveFunction{MOI.ScalarAffineFunction{Float64}}(),
    MOI.ScalarAffineFunction(MOI.ScalarAffineTerm.(c, x), 0.0))
MOI.set(optimizer, MOI.ObjectiveSense(), MOI.MAX_SENSE)

# Ограничение по весу
MOI.add_constraint(optimizer,
    MOI.ScalarAffineFunction(MOI.ScalarAffineTerm.(w, x), 0.0),
```

```
MOI.LessThan(C))

# Бинарные переменные
for x_i in x
    MOI.add_constraint(optimizer, x_i, MOI.ZeroOne())
end

MOI.optimize!(optimizer)
```

Пояснение: Код формулирует задачу о рюкзаке как задачу смешанно-целочисленного линейного программирования. Нотация `MOI.ScalarAffineTerm.(c, x)` использует broadcast-синтаксис Julia для создания вектора линейных членов. Решение `sol = MOI.get(optimizer, MOI.VariablePrimal(), x)` возвращает `[1.0, 1.0, 1.0]` с целевым значением 6.0, что соответствует выбору всех предметов при заданной вместимости.